Lab Task 1: Stock Price Monitor using AVL Tree

You are building a **real-time stock price monitoring system** for a trading platform.
The platform continuously receives updates about stocks — new ones being added, existing ones changing price, and occasionally being removed.

The system must support:

- **Fast search and updates** by stock ID.
- **Automatic rebalancing** to ensure performance remains consistent even under thousands of updates.
- **Analytical queries** such as getting all stocks within a price range or identifying the highest and lowest priced stocks.

Since stock data arrives dynamically and unpredictably, the system uses an **AVL Tree** keyed by `stockID` to maintain **O(log n)** efficiency for insertions, deletions, and lookups.

Data Model

Each node in the AVL tree represents one stock record.

```
struct Stock {
    int stockID;          // Unique key for stock (e.g., 101)
    char name[50];        // Stock name
    float price;          // Current stock price
};

struct Node {
    Stock data;
    Node* left;
    Node* right;
    int height;
};
```

Functional Requirements

1. **Insertion — `addStock()`**

- Inserts a new stock into the AVL tree using `stockID` as the key.
- Duplicate IDs are **not allowed** — reject duplicates and return `false`.
- Automatically rebalances the tree after insertion.

2. **Update — `updatePrice()`**
   - Updates the `price` of an existing stock.
   - Must preserve the AVL property.
   - If stock not found, return `false`.
3. **Deletion — `removeStock()`**
   - Removes a stock node by `stockID`.
   - Properly rebalances the tree after deletion.
4. **Find — `findStock()`**
   - Returns a pointer to the stock record matching the ID.
   - Returns `nullptr` if not found
5. **Range Query — `getStocksInPriceRange()`**
   - Returns all stocks whose `price` lies in `[minPrice, maxPrice]`.
   - Output via dynamically allocated array (created with `new`), and sets `outCount` to the number of results.
6. **Highest/Lowest Queries**
   - `getHighestPricedStock()` returns the stock with the maximum price.
   - `getLowestPricedStock()` returns the stock with the minimum price.
   - If the tree is empty, return a dummy Stock with `stockID = -1`.
7. **Diagnostics**
   - `isBalanced()` : verifies AVL property (height difference ≤ 1 for all nodes).

- `getHeight()` : returns the tree's height.
- `getTotalStocks()` : returns the total number of stocks currently stored.

8. **Display**
   - `displayInorder()` : prints stock IDs in ascending order for debugging.

Test Cases

```
#include "gtest/gtest.h"
#include "StockAVL.h"

// 1. Insert and Find Stock
TEST(StockAVLTest, AddAndFindStock) {
    StockAVL avl;
    EXPECT_TRUE(avl.addStock(101, "Tesla", 750.5));
    EXPECT_TRUE(avl.addStock(102, "Apple", 180.2));

    Stock* s = avl.findStock(101);
    ASSERT_TRUE(s != nullptr);
    EXPECT_STREQ(s->name, "Tesla");
    EXPECT_FLOAT_EQ(s->price, 750.5);
}

// 2. Reject Duplicate Stock IDs
TEST(StockAVLTest, RejectDuplicateStockIDs) {
    StockAVL avl;
    EXPECT_TRUE(avl.addStock(100, "Microsoft", 320.1));
    EXPECT_FALSE(avl.addStock(100, "Meta", 240.0));
    EXPECT_EQ(avl.getTotalStocks(), 1);
}

// 3. Update Existing Stock Price
TEST(StockAVLTest, UpdateStockPrice) {
    StockAVL avl;
    avl.addStock(1, "NVIDIA", 100.0);
    EXPECT_TRUE(avl.updatePrice(1, 120.0));

    Stock* s = avl.findStock(1);
```

```
    ASSERT_TRUE(s != nullptr);
    EXPECT_FLOAT_EQ(s->price, 120.0);
}

// 4. Update Non-Existing Stock Fails
TEST(StockAVLTest, UpdateNonExistingStock) {
    StockAVL avl;
    avl.addStock(5, "Intel", 35.0);
    EXPECT_FALSE(avl.updatePrice(10, 90.0)); // no stock with ID 10
}

// 5. Auto-Balancing After Multiple Inserts (LR Rotation)
TEST(StockAVLTest, AutoBalancingAfterInsert) {
    StockAVL avl;
    avl.addStock(30, "A", 10.0);
    avl.addStock(10, "B", 20.0);
    avl.addStock(20, "C", 30.0); // triggers LR rotation
    EXPECT_TRUE(avl.isBalanced());
}

// 6. Delete Stock and Maintain Balance
TEST(StockAVLTest, DeleteAndRebalance) {
    StockAVL avl;
    avl.addStock(50, "A", 10.0);
    avl.addStock(40, "B", 15.0);
    avl.addStock(60, "C", 20.0);

    EXPECT_TRUE(avl.removeStock(40));
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_EQ(avl.getTotalStocks(), 2);
}

// 7. Remove Non-Existing Stock
TEST(StockAVLTest, RemoveNonExistingStock) {
    StockAVL avl;
    avl.addStock(10, "A", 100.0);
    EXPECT_FALSE(avl.removeStock(99));
    EXPECT_EQ(avl.getTotalStocks(), 1);
}
```

```
// 8. Find Highest and Lowest Priced Stocks
TEST(StockAVLTest, HighestLowestStock) {
    StockAVL avl;
    avl.addStock(1, "X", 50.0);
    avl.addStock(2, "Y", 100.0);
    avl.addStock(3, "Z", 75.0);

    Stock high = avl.getHighestPricedStock();
    Stock low  = avl.getLowestPricedStock();

    EXPECT_STREQ(high.name, "Y");
    EXPECT_STREQ(low.name, "X");
}

// 9. Get Stocks in Price Range
TEST(StockAVLTest, StocksInPriceRange) {
    StockAVL avl;
    avl.addStock(1, "A", 10.0);
    avl.addStock(2, "B", 50.0);
    avl.addStock(3, "C", 70.0);
    avl.addStock(4, "D", 100.0);

    int count = 0;
    Stock* results = avl.getStocksInPriceRange(40.0, 80.0, count);
    EXPECT_EQ(count, 2); // Stocks B and C
    EXPECT_TRUE(results[0].price >= 40.0 && results[0].price <= 80.0);
    EXPECT_TRUE(results[1].price >= 40.0 && results[1].price <= 80.0);
    delete[] results;
}

// 10. Tree Height After Multiple Insertions
TEST(StockAVLTest, HeightAfterInsertions) {
    StockAVL avl;
    for (int i = 1; i <= 15; ++i)
        avl.addStock(i, "S", (float)(i * 10));
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_LT(avl.getHeight(), 8);
}

// 11. Deletion Sequence Maintains Balance
```

```
TEST(StockAVLTest, SequentialDeletionsBalanced) {
    StockAVL avl;
    for (int i = 1; i <= 10; ++i)
        avl.addStock(i, "P", (float)(i * 5));
    for (int i = 2; i <= 10; i += 2)
        avl.removeStock(i);
    EXPECT_TRUE(avl.isBalanced());
}

// 12. Delete All Stocks
TEST(StockAVLTest, DeleteAllStocks) {
    StockAVL avl;
    avl.addStock(10, "A", 20.0);
    avl.addStock(20, "B", 30.0);
    avl.addStock(30, "C", 40.0);

    avl.removeStock(10);
    avl.removeStock(20);
    avl.removeStock(30);

    EXPECT_EQ(avl.getTotalStocks(), 0);
    EXPECT_TRUE(avl.isBalanced());
}

// 13. Empty Tree Highest/Lowest
TEST(StockAVLTest, EmptyTreeEdgeCase) {
    StockAVL avl;
    Stock high = avl.getHighestPricedStock();
    Stock low = avl.getLowestPricedStock();
    EXPECT_EQ(high.stockID, -1);
    EXPECT_EQ(low.stockID, -1);
}
```

Lab Task 2: Course Scheduling

Scenario:

Build an **AVL Tree** based course scheduling system where each course is keyed by its **startTime** (an `int` representing minutes since midnight). The AVL property must be preserved after every insertion and deletion. Beyond correctness of the tree, implement efficient domain-specific queries such as finding overlapping courses and computing free time slots.

The registrar needs a scheduler that stores course slots ordered by their start time and supports fast queries and updates. The system must:

- Keep course records balanced so queries remain `O(log n)`.
- Allow finding all courses that conflict with (overlap) a given time interval.
- Compute all free time intervals (gaps) between courses during a day.
- Support insertion and deletion of courses and always maintain AVL balance.
- Provide diagnostic functions to verify AVL-property and tree height.

Use an **AVL tree keyed by `startTime`**. If two courses have the same `startTime`, reject the insertion (start times must be unique for simplicity). Times are integers in minutes in range `[0, 1440)`; courses have `startTime < endTime`. The day runs from `0` (00:00) to `1440` (24:00).

Data Model and API

```
// Course record
struct Course {
    int courseID;      // unique course identifier
    int startTime;     // minutes since midnight [0, 1440)
    int endTime;       // minutes since midnight, startTime < endTime
    char title[128];   // fixed-size C-string for simplicity
};

// AVL node
struct Node {
    Course data;
```

```
    Node* left;
    Node* right;
    int height;
};

// Scheduler API (member functions of CourseAVL class)
class CourseAVL {
public:
    CourseAVL();
    ~CourseAVL();

    // Core operations
    bool addCourse(int courseID, int startTime, int endTime, const char*
title);
    bool removeCourseByStart(int startTime);     // remove by key
    Course* findCourseByStart(int startTime);   // returns pointer or
nullptr

    // Queries
    // findOverlaps returns dynamic array (allocated with new[]) and sets
outCount.
    // Caller is responsible for deleting the returned array with
delete[].
    Course* findOverlaps(int qStart, int qEnd, int &outCount);

    // getFreeSlots returns an array of pairs (start, end) as Course-like
objects
    // where courseID = -1 and title empty; outCount set to number of
gaps.
    Course* getFreeSlots(int &outCount);

    // Diagnostics
    int getTotalCourses();
    int getHeight();
    bool isBalanced(); // verifies AVL property across all nodes

    // Utility: pretty print (inorder) - prints to stdout
    void displayInorder();

    // (internal helpers are allowed)
```

```
};
```

Requirements & Constraints

1. **AVL correctness**
    - After every `addCourse` and `removeCourseByStart`, the tree must be height-balanced (balance factor ∈ {-1,0,1} for every node).
    - Rotations (LL, RR, LR, RL) must update `height` correctly.
2. **Key uniqueness**
    - `startTime` is the key. If `addCourse` receives a `startTime` that already exists, it should reject the insert and return `false`.
3. **Validation**
    - Reject invalid times (e.g., `startTime >= endTime`, or times outside `[0,1440]`).
4. **Overlap definition**
    - Two intervals `[a,b)` and `[c,d)` overlap if `max(a,c) < min(b,d)`.
    - Example: `[9:00,10:00)` and `[9:30,10:30)` overlap.
5. **Free slot computation**
    - The day is `[0, 1440)`. Free slots are maximal intervals not covered by any course.
    - Adjacent courses (endTime == next startTime) produce no gap between them.
    - If there are no courses, a single free slot `[0,1440)` is returned.
6. **Memory**

- No STL containers. Dynamic arrays can be returned via `new` with size communicated via `outCount`.
- Ensure no memory leaks (delete nodes in destructor).

7. **Efficiency targets**
   - `addCourse`, `removeCourseByStart`, `findCourseByStart`: `O(log n)` average/worst (due to AVL).
   - `findOverlaps(qStart, qEnd)`: should be `O(k + log n)` where `k` is number of overlapping courses (use interval pruning).
   - `getFreeSlots`: `O(n)` (inorder traversal once).

Implementation Notes:

1. Store courses keyed by `startTime`.
2. For `findOverlaps`, perform a modified inorder or recursive search:
   - If a node's `startTime` >= `qEnd`, only search left subtree.
   - If a node's `endTime` <= `qStart`, only search right subtree.
   - Otherwise, the node overlaps; record it and search both subtrees.
3. For `getFreeSlots`:
   - Traverse nodes in inorder; track `previousEnd` starting at 0.
   - For each node visited with interval `[s,e)`, if `s > previousEnd` add gap `[previousEnd, s)`, then set `previousEnd = max(previousEnd, e)`.
   - After traversal, if `previousEnd < 1440`, add final gap `[previousEnd, 1440)`.
4. For `isBalanced`, compute heights bottom-up and verify balance factor for each node (return false on any violation).
   - Carefully update `height` in rotations and in insert/delete rebalancing code.
5. For deletion of a node with two children, replace with inorder successor (min of right subtree), then rebalance upwards.

Test Cases

```
TEST(CourseSchedulerTest, InsertAndFind) {
    CourseAVL sched;
    bool ok;

    ok = sched.addCourse(101,  540,  600, "CS101"); // 9:00-10:00
    EXPECT_TRUE(ok);
    ok = sched.addCourse(102,  600,  660, "CS102"); // 10:00-11:00
    EXPECT_TRUE(ok);

    Course* c = sched.findCourseByStart(540);
    ASSERT_TRUE(c != nullptr);
    EXPECT_EQ(c->courseID, 101);
    EXPECT_EQ(c->startTime, 540);
    EXPECT_EQ(c->endTime, 600);

    EXPECT_EQ(sched.getTotalCourses(), 2);
    EXPECT_TRUE(sched.isBalanced());
}
TEST(CourseSchedulerTest, RejectDuplicatesAndInvalid) {
    CourseAVL sched;
    bool ok;
    ok = sched.addCourse(201, 480, 540, "Math"); // valid
    EXPECT_TRUE(ok);
    ok = sched.addCourse(202, 480, 520, "Physics"); // duplicate start
    EXPECT_FALSE(ok); // rejected

    ok = sched.addCourse(203, 700, 700, "ZeroLength"); // invalid
(start==end)
    EXPECT_FALSE(ok);

    ok = sched.addCourse(204, -10, 50, "NegativeStart"); // invalid
    EXPECT_FALSE(ok);
}
TEST(CourseSchedulerTest, FindOverlaps) {
    CourseAVL sched;
    sched.addCourse(1,  540, 600, "A"); // 9:00-10:00
    sched.addCourse(2,  570, 630, "B"); // 9:30-10:30
    sched.addCourse(3,  660, 720, "C"); // 11:00-12:00
    sched.addCourse(4,  600, 660, "D"); // 10:00-11:00
```

```
    int count = 0;
    Course* overlaps = sched.findOverlaps(580, 615, count); // 9:40-10:15

    // Expected overlapping courses: 1 (9:00-10:00), 2 (9:30-10:30), 4
(10:00-11:00) -> 3 items
    EXPECT_EQ(count, 3);

    // Check that returned items are indeed overlapping and have proper
times
    for (int i = 0; i < count; ++i) {
        EXPECT_TRUE(max(overlaps[i].startTime, 580) <
min(overlaps[i].endTime, 615));
    }
    delete[] overlaps;
}
TEST(CourseSchedulerTest, GetFreeSlots) {
    CourseAVL sched;
    // Day: 0-1440
    sched.addCourse(10,  480, 540, "Morning");    // 8:00-9:00
    sched.addCourse(11,  600, 660, "LateMorning"); // 10:00-11:00
    sched.addCourse(12,  720, 780, "Noon");       // 12:00-13:00

    int count = 0;
    Course* gaps = sched.getFreeSlots(count);

    // Gaps expected:
    // [0,480), [540,600), [660,720), [780,1440)  --> 4 gaps
    EXPECT_EQ(count, 4);

    // Check each gap is valid (courseID == -1 indicates gap)
    EXPECT_EQ(gaps[0].courseID, -1);
    EXPECT_EQ(gaps[0].startTime, 0);
    EXPECT_EQ(gaps[0].endTime, 480);

    EXPECT_EQ(gaps[3].startTime, 780);
    EXPECT_EQ(gaps[3].endTime, 1440);

    delete[] gaps;
}
```

```
TEST(CourseSchedulerTest, DeletionCausesRotation) {
    CourseAVL sched;
    // Insert in order that will create skew and require rotations when
deleting
    sched.addCourse(1,  300, 330, "A");
    sched.addCourse(2,  200, 230, "B");
    sched.addCourse(3,  100, 130, "C"); // causes imbalance (left-left)
    EXPECT_TRUE(sched.isBalanced());

    // Remove the left-most to force rebalancing on path
    bool ok = sched.removeCourseByStart(100);
    EXPECT_TRUE(ok);
    EXPECT_TRUE(sched.isBalanced());
    EXPECT_EQ(sched.getTotalCourses(), 2);
}
TEST(CourseSchedulerTest, FullDayAndBackToBack) {
    CourseAVL sched;
    // Full day course
    bool ok = sched.addCourse(50, 0, 1440, "FullDay");
    EXPECT_TRUE(ok);

    int count = 0;
    Course* gaps = sched.getFreeSlots(count);
    // No gaps expected
    EXPECT_EQ(count, 0);
    delete[] gaps;

    // Remove full day
    EXPECT_TRUE(sched.removeCourseByStart(0)); // removes startTime == 0
node

    // Back-to-back courses
    sched.addCourse(60, 480, 540, "A"); // 8:00-9:00
    sched.addCourse(61, 540, 600, "B"); // 9:00-10:00 (back-to-back)
    count = 0;
    gaps = sched.getFreeSlots(count);
    // Expect gaps: [0,480), [600,1440) -> 2 gaps
    EXPECT_EQ(count, 2);
```

```
    delete[] gaps;
}
```